

Edit by chaoweic

复杂度速查

图论

算法	时间复杂度	空间复杂度
BFS（无权最短路）	$O(V + E)$	$O(V)$
网格 BFS	$O(n \times m)$	$O(n \times m)$
0-1 BFS	$O(V + E)$	$O(V)$
双向 BFS	$O(b^{(d/2)})$	$O(b^{(d/2)})$
Dijkstra	$O((V + E) \log V)$	$O(V)$
SPFA	$O(VE)$ （最坏）	$O(V)$
二分图判定（染色 BFS/DFS）	$O(V + E)$	$O(V)$
Khan 拓扑排序	$O(V + E)$	$O(V)$
Tarjan 强连通分量	$O(V + E)$	$O(V)$
Kruskal 最小生成树	$O(E \log E)$	$O(V + E)$
树上 DFS 遍历	$O(V)$	$O(V)$
DFS 排列枚举	$O(n! \cdot n)$	$O(n)$
DFS 子集生成	$O(2^n)$	$O(n)$

数据结构

数据结构	建表 / 插入	查询 / 更新	空间
并查集 (DSU)	$O(\alpha(n))$ 均摊	$O(\alpha(n))$ 均摊	$O(n)$

数据结构	建表 / 插入	查询 / 更新	空间
ST 表	$O(n \log n)$	$O(1)$	$O(n \log n)$
树状数组 (BIT)	$O(n)$ 建表 / $O(\log n)$ 单点	$O(\log n)$	$O(n)$
线段树（含懒标记）	$O(n)$ 建表	$O(\log n)$ 区间	$O(n)$
优先队列（多元素）	$O(\log n)$ push / $O(1)$ top	$O(\log n)$ pop	$O(n)$

字符串

算法 / 结构	预处理	单次查询	空间
字符串哈希	$O(n)$	$O(1)$ 子串哈希 / $O(\log n)$ LCP	$O(n)$
KMP	$O(m)$	$O(n + m)$ 首次匹配	$O(m)$
前缀函数 (π)	$O(n)$	$O(n + m)$ 全部匹配	$O(n)$
Manacher	$O(n)$	$O(n)$	$O(n)$
Trie 树	$O(s)$ 插入	$O(s)$ 查找	$O(\text{总字符数} \times \Sigma)$
AC 自动机	$O(\text{总模式串长度} \times \Sigma)$	$O(t + \text{匹配数})$	$O(\text{总字符数} \times \Sigma)$

数学

算法	时间复杂度	空间复杂度
线性筛	$O(n)$	$O(n)$
质因数分解	$O(\log x)$	$O(1)$
快速幂（模质数）	$O(\log b)$	$O(1)$
逆元（费马小定理）	$O(\log \text{MOD})$	$O(1)$
矩阵快速幂	$O(m^3 \log k)$	$O(m^2)$

算法	时间复杂度	空间复杂度
异或线性基	$O(B)$ 插入 / $O(B)$ 查询	$O(B)$
排列组合数（模）	$O(n)$ 预处理 / $O(1)$ 查询	$O(n)$
容斥原理	$O(2^k \cdot k)$	$O(k)$
卡特兰数	$O(1)$ （预处理后）	$O(1)$
斯特林数（第二类）	$O(nk)$	$O(nk)$
扩展欧几里得	$O(\log \min(a,b))$	$O(\log \min(a,b))$

Debug 错误清单

通用

- **整数溢出**：int 相乘超过 $2^{31}-1$ （约 $2.1e9$ ），两个 $1e5$ 相乘就可能溢出； $1e9$ 平方直接爆 long long 中间结果都要留意
- **数组开小或开大**：题目给的是 $1e5$ ，但存图/离散化后规模翻倍（比如加了虚点、加了反向边），数组要按实际使用规模开，不是题目原始规模
- **数组越界**：尤其是下标从1开始还是0开始不统一，导致读写越界但没有RE（未定义行为，可能没报错但结果错）
- **多组数据未清空**：这是ICPC最经典的送命题——memset / clear / fill 漏了任何一个全局数组、vector、map，前一组数据的残留会污染下一组
- **未初始化变量**：局部变量没赋初值就使用，本地跑对是因为内存刚好是0，评测机上不一定
- **负数取模**：C++ 中负数取模结果可能是负数， $(a \% p + p) \% p$ 才安全
- **浮点数比较用 ==**：应该用 $\text{fabs}(a-b) < \text{eps}$
- **除法/取模中的0**：分母为0、模数为0没有特判
- **递归爆栈**：DFS 递归层数过深（链状图/树），本地栈大评测机栈小，容易本地过、提交RE，需要改迭代或线程扩栈
- **值传递 vs 引用传递搞反**：大对象（vector / struct）值传递导致隐性 $O(n)$ 拷贝，可能是隐蔽的TLE源
- **迭代器失效**：vector 在遍历中 erase / push_back 导致迭代器失效
- **自定义比较函数不满足严格弱序**：sort 的 cmp 如果不满足传递性/反对称性，可能直接崩溃或死循环

- **变量名与库冲突**：using namespace std 后自己定义 y1、next、count 等和标准库同名的变量/函数名
- **自增自减在同一表达式多次使用**：如 `a[i++] = i++`，属于未定义行为
- **cin / cout 没关同步却还用 scanf / printf 混用**：导致IO顺序错乱或变慢

数据结构类

并查集

- 路径压缩写法错误，导致 find 返回的不是最新根
- 初始化 `parent[i]=i` 忘记（尤其多组数据时忘记重新初始化）
- 按秩合并/按size合并条件写反
- 带权并查集：合并时关系值（相对偏移）计算顺序或符号搞反

线段树

- pushdown 顺序错误：应该在递归进入子树**之前**下传 lazy
- mid 计算 $(l+r)/2$ 在 `l+r` 较大时可能溢出（一般竞赛范围不至于，但要留意）
- 区间赋值 tag 与区间加 tag 同时存在时，优先级/清空顺序处理错误（一般"先乘后加"或明确谁覆盖谁）
- pushup 忘记调用，导致父节点信息没更新
- 权值线段树/动态开点线段树内存没有按需动态申请，一次性开满导致 MLE

树状数组

- lowbit 写错（应为 `x & (-x)`）
- 下标从1开始，代码里误从0开始导致死循环（`lowbit(0)=0`）
- 区间修改+区间查询用的差分公式记错

单调栈 / 单调队列

- 弹出条件的等号使用（`>=` 还是 `>`）会导致相等元素处理方式不同，进而重复计算或漏算
- 存的是下标还是值搞混，导致计算距离/宽度时用错

图论类

- 无向图建边忘记加反向边，或者加了两次导致边数变成两倍影响其他逻辑

- **Dijkstra 用在负权图**：结果错误但不报错，容易查很久才发现根源是"图有负权"这个前提就不满足
- **SPFA 没有判负环**：死循环或者一直不收敛，需要用"入队次数超过n"来判负环
- 堆优化 Dijkstra 的 vis 标记位置不对：应该在**出队时**判断并跳过，而不是入队时标记
- Floyd 三层循环顺序写错（k 必须是最外层循环，这个错误极其隐蔽）
- Kruskal 忘记先按边权排序
- 拓扑排序时忘记处理有多个连通分量的情况（图不连通）
- Tarjan 求强连通分量：dfn / low 更新时机、栈的进出时机稍有偏差整个算法就错
- 二分图匹配（匈牙利算法）：每次尝试新的匹配前，vis 数组没有清空
- 树上倍增求LCA： $\log_2(n)$ 数组大小开小了（应为 $\log_2(n)+1$ 甚至更保守）
- 树链剖分：dfs 两次的顺序、top / son（重儿子）计算错误

动态规划类

- 状态定义模糊，导致转移方程和实际想要表达的意思不一致（写代码前一定要能用一句话说清楚 dp[i][j] 表示什么）
- 边界初始化错误：该初始化成 0 还是 $-\infty / \infty$ 没想清楚（求最大值初始化成 $-\infty$ ，求最小值初始化成 ∞ ，这个反了整个DP都错）
- **01背包用一维数组忘记倒序遍历**，导致变成了完全背包的效果
- 完全背包正序、01背包倒序，两者弄反
- 多重背包二进制分组的边界计算错误（最后剩余数量没处理好）
- 区间DP：必须按**区间长度从小到大**枚举，而不是直接嵌套 l, r，否则用到了还没计算出来的状态
- 树形DP：子树的状态必须先算完（后序遍历）才能更新父节点，顺序反了会用到未计算的子树状态
- 状压DP：位运算写反（比如判断某一位是否被选中的位运算符用错），或者子集枚举写错（for (int j = i; j; j = (j-1)&i) 这类写法很容易写错终止条件）
- 记忆化搜索初始值设置成一个可能和合法答案冲突的哨兵值（比如用 -1 表示"未计算"，但答案本身可能就是 -1）
- 滚动数组：下标忘记 %2 或者 &1，导致新旧状态互相覆盖

数论 / 数学类

- 用费马小定理求逆元时，模数不是质数（费马小定理要求模数为质数，否则要用扩展欧几里得）
- 快速幂指数为负数或为0时没有特判
- 组合数取模：模数不是质数时不能直接用阶乘逆元公式，需要卢卡斯定理或其他方法

- 中国剩余定理：模数不两两互质时，普通CRT公式不适用，需要扩展CRT
- **两个 long long 范围内的数相乘依然可能溢出**（比如两个接近 $1e18$ 的数相乘），这种情况需要 `__int128`
- 除法取模需要转换成乘逆元，不能直接对分子分母分别取模再相除
- gcd 函数处理 `gcd(0,0)` 或负数输入时的边界情况

字符串类

- KMP 的 `next / fail` 数组下标定义从0开始还是从1开始要和主逻辑保持一致，否则匹配位置全部错位
- 字符串 Hash 只用单哈希容易被构造数据卡掉（生日攻击），建议双哈希+大质数模数
- Trie 树内存没有预先估算够大，或者动态开点时索引计算错误

计算几何类

- 浮点数精度问题：`eps` 设置不合理（太大导致精度损失，太小又对付不了实际误差）
- 用 `int` 计算叉积/点积：坐标范围较大时中间结果溢出，应提前转 `long long` 或 `double`
- 极角排序时 `atan2` 的象限判断不完整，导致排序结果在跨象限处出错

输入输出 / 多组数据类

- 没有关闭 `cin / cout` 同步（`ios::sync_with_stdio(false); cin.tie(0);`）导致大数据量下TLE
- **多组数据的清空是重灾区**：任何全局 `vector`、图的邻接表、`dp` 数组、标记数组，只要漏清空一个就会在后续数据组里出诡异的错误，而且这种错误往往具有随机性（取决于上一组数据残留的具体数值），很难复现和定位
- 读入格式与题目描述不完全匹配（多余空格、连续多个空格、行末换行符处理）
- `scanf` 格式符与变量类型不匹配（比如 `long long` 用了 `%d` 读入，静默截断不报错）

隐藏的复杂度/性能

- 每次查询都重新做一次 $O(n)$ 预处理，本该预处理一次，结果总复杂度从 $O(n)$ 变成 $O(nq)$
- `vector` 频繁 `push_back` 没有提前 `reserve`，导致频繁扩容拷贝
- 大量数据输出时用 `endl`（每次刷新缓冲区）而不是 `'\n'`，看起来只是习惯问题但数据量大时能差出几倍时间
- 字符串在循环里用 `+=` 拼接导致隐性 $O(n^2)$ （应该用 `string` 的 `reserve` 或者 `stringstream` / 先收集再一次性拼接）


```
#!/usr/bin/env python3
```

```
"""
```

```
python gen.py
```

边界情况（ $n=1$ 、全部相同值、最大值最小值）要有一定概率被覆盖到，不要总是生成"中庸"的随机数据

```
"""
```

```
import random
```

```
import sys
```

```
import time
```

```
# 如果传入了参数就用固定seed（方便复现某次失败），否则用当前时间保证每次不同
```

```
if len(sys.argv) > 1:
```

```
    random.seed(int(sys.argv[1]))
```

```
else:
```

```
    random.seed(time.time_ns())
```

```
def rand_int(lo, hi):
```

```
    return random.randint(lo, hi)
```

```
def gen_array(n, lo, hi):
```

```
    """生成长度为n、值域在[lo,hi]的数组"""
```

```
    return [rand_int(lo, hi) for _ in range(n)]
```

```
def gen_permutation(n):
```

```
    """生成1~n的随机排列"""
```

```
    p = list(range(1, n + 1))
```

```
    random.shuffle(p)
```

```
    return p
```

```
def gen_tree_edges(n):
```

```
    """
```

生成一棵 n 个节点的树（ $n-1$ 条边），节点编号 $1\sim n$

做法：对每个节点 $i(2\sim n)$ ，随机选一个比它编号小的节点作为父节点

这样保证一定是一棵合法的树，不会成环

```
    """
```

```
    edges = []
```

```
    for i in range(2, n + 1):
```

```
        parent = rand_int(1, i - 1)
```

```
        edges.append((parent, i))
```

```
random.shuffle(edges) # 打乱边的顺序，避免"父节点编号总是更小"
return edges
```

```
def gen_graph_edges(n, m, allow_multi_edge=False, allow_self_loop=False, directed=False):
    """
    生成n个点m条边的随机图
    m 不要超过 n*(n-1)/2（无向无重边情况下的上限），否则会死循环
    """
    edges = []
    seen = set()
    attempts = 0
    while len(edges) < m and attempts < m * 50:
        attempts += 1
        u, v = rand_int(1, n), rand_int(1, n)
        if not allow_self_loop and u == v:
            continue
        key = (u, v) if directed else tuple(sorted((u, v)))
        if not allow_multi_edge and key in seen:
            continue
        seen.add(key)
        edges.append((u, v))
    return edges
```

```
def gen_string(n, alphabet="ab"):
    """生成长度为n、字符集为alphabet的随机字符串"""
    return "".join(random.choice(alphabet) for _ in range(n))
```

```
def maybe_edge_case(n_max):
    """
    有一定概率强制返回边界情况的n，而不是纯随机
    边界情况往往是代码最容易挂的地方，对拍时应该主动多测这些
    """
    r = random.random()
    if r < 0.15:
        return 1
    elif r < 0.25:
        return 2
    elif r < 0.35:
        return n_max
    else:
```

```

    return rand_int(1, n_max)

def main():
    N_MAX = 10    # 数据规模，对拍时开小方便肉眼调试，确认没问题后再逐步调大
    VAL_MAX = 100

    n = maybe_edge_case(N_MAX)

    print(n)
    arr = gen_array(n, 1, VAL_MAX)
    print(*arr)

    # ---- 如果题目是图论/树，换成下面这种输出方式，把上面数组部分删掉 ----
    #
    # n = maybe_edge_case(N_MAX)
    # edges = gen_tree_edges(n)
    # print(n)
    # for u, v in edges:
    #     print(u, v)
    #
    # 或者一般图:
    # n = rand_int(1, N_MAX)
    # m = rand_int(0, min(n * (n - 1) // 2, 20))
    # edges = gen_graph_edges(n, m)
    # print(n, m)
    # for u, v in edges:
    #     print(u, v)

    # ---- 如果题目是字符串，参考： ----
    #
    # n = maybe_edge_case(N_MAX)
    # s = gen_string(n, alphabet="ab")
    # print(n)
    # print(s)

    # -----

if __name__ == "__main__":
    main()

```

```
@echo off
chcp 65001 >nul
set cnt=0

:loop
set /a cnt+=1
echo 测试 #%cnt%

python gen.py > input.txt
brute.exe < input.txt > 暴力输出.txt
solve.exe < input.txt > 实际输出.txt

fc 暴力输出.txt 实际输出.txt > nul
if errorlevel 1 (
    echo [x] 找到差异! 输入如下:
    type input.txt
    echo.
    echo 暴力输出:
    type 暴力输出.txt
    echo 你的输出:
    type 实际输出.txt
    pause
    goto loop
)
echo [v] 通过
goto loop
```

```
#!/bin/bash
cnt=0
while true; do
    cnt=$((cnt + 1))
    echo "测试 #${cnt}"

    python3 gen.py > input.txt
    ./brute < input.txt > 暴力输出.txt
    ./solve < input.txt > 实际输出.txt

    if ! diff -q 暴力输出.txt 实际输出.txt > /dev/null; then
        echo "找到差异！输入如下："
        cat input.txt
        echo ""
        echo "暴力输出："
        cat 暴力输出.txt
        echo "你的输出："
        cat 实际输出.txt
        exit 1
    fi
    echo "通过"
done
```